Profesor: Sr. Rolando Montero
rmontero@colegiogermania.cl

Nombre : ..... Curso: ..... Fecha: .....

Estructura del PyGame

Objetivos	Utilizar conocimientos de pygame, para comprender el comportamiento de un video juego.
Contenidos	Sistema numérico de diferentes bases.
Habilidades	Aplicación – Análisis – evaluación.

Para crear un video juego se deben establecer tres ámbitos los que a continuación se describen.

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

# Autor:
# Fecha:
# Extracto:
# Licencia:

# --------------------------
# Importación de los módulos
# --------------------------
import pygame
from pygame.locals import *
import sys
import math

# -----------
# Constantes
# -----------

# ------------------------------
# Clases
# ------------------------------

# ------------------------------
# Clases
# ------------------------------

# ------------------------------
# Función principal del juego
# ------------------------------

def main():
    pygame.init()
    … … continuación

if __name__ == "__main__":
    main()
```

En esta parte trataremos los siguientes puntos:

- Como crear una ventana
- El bucle principal de un videojuego (o como saber cuándo hay que terminar el juego)
- Usar una imagen como fondo
- Mostrar una imagen y mover la imagen en la pantalla

Paso 1: Creando una ventana

Primero Importamos los módulos y definimos dos variables globales (por metodología se suelen poner en mayúsculas) que son resolución de la pantalla del juego.

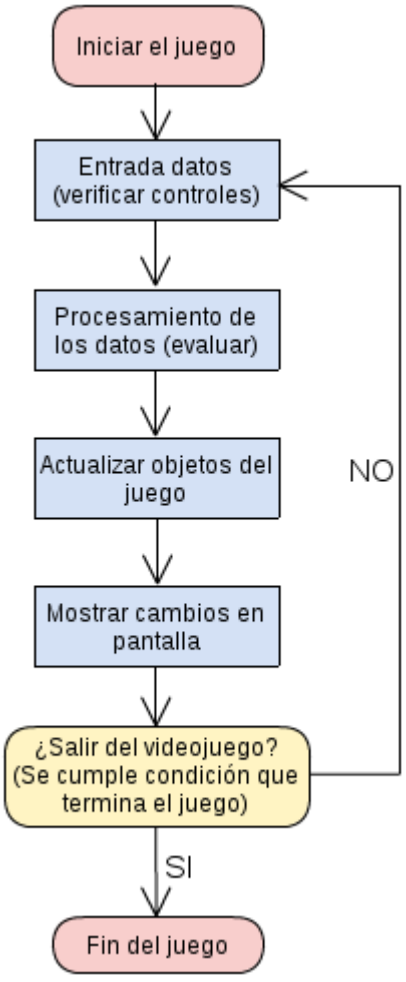
Luego hay que crear la ventana, para ello dentro de la función main() usamos pygame.display.set_mode y pygame.display.set_caption, que establecen el tamaño de la ventana y el título de esta respectivamente:

```
def main():
    pygame.init()
    # creamos la ventana y le indicamos un titulo:
    screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
    pygame.display.set_caption("título de la ventana")


if __name__ == "__main__":
    main()
```

Paso 2: el bucle principal del juego

Los juegos generalmente ejecutan un bucle infinito (su bucle principal), lo cual permite que el juego continúe corriendo hasta la que se cumpla una condición que lo termine (como por ejemplo que el jugador pierda todas sus vidas o se rinda), más o menos lo que muestra este diagrama:



En nuestro caso el bucle principal se continuará ejecutando hasta que el jugador cierre el ventana (o sea que se quede esperando hasta que se haga click en el botón para cerrar la ventana)

Al código siguiente hay que agregar.

```
# el bucle principal del juego
    while True:
        # Posibles entradas del teclado y mouse
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                sys.exit()
```

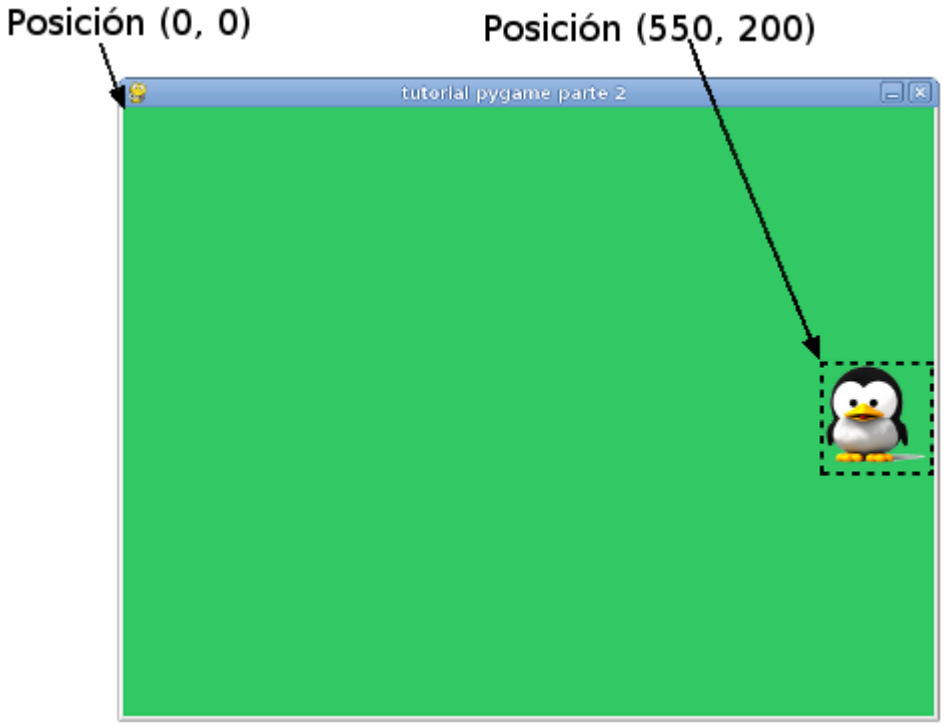
Lo que hace es simple, una vez dentro del bucle, recorre la lista que devuelve pygame.event.get() (es una lista con todos los eventos que registra pygame, como por ejemplo cuando uno presiona una tecla), si dentro de esa lista esta el evento pygame.QUIT (o sea cerrar la ventana) el programa ejecuta la orden sys.exit() (que termina la ejecución del programa y es parte del modulo sys, por eso lo importamos), en caso contrario (que aun no se produzca ese evento) seguirá esperando.

Paso 3: Cargando el fondo y una imagen

Primero para cargar las imágenes usamos pygame.image.load(), con ello se crea un objeto que contiene la superficie de la imagen (aun no la muestra). Luego de esto hay que indicar las posiciones de esta imagen, para lo cual se usa blit(imagen, (coordenada_x, coordenada_y))

Puedes pensar en la ventana de pygame como un plano en 2 ejes, con una pequeña diferencia: el origen (coordenadas 0,0) se encuentra en la esquina superior izquierda de la ventana. Por lo tanto, en valor de las coordenadas en el eje x (horizontal) va aumentando (y es positivo) mientras se avanza a la derecha, mientras que en el eje y (vertical) el valor va aumentando (y es positivo) a medida que se avanza hacia abajo (o sea lo contrario de lo que se acostumbra, para el eje y).

Para que quede más claro, un esquema en que muestra en qué posición cargaremos las imágenes (Nota: pygame siempre usa como referencia la esquina superior izquierda de las imágenes).



Ahora que ya sabemos sobre coordenadas, vamos a insertar un fondo y una imagen en nuestra ventana, usaremos las siguientes:

Fondo:



fondo.jpg

imagen a mostrar:



tux.png

```
    # cargamos el fondo y una imagen (se crea objetos "Surface")
    fondo = pygame.image.load("fondo.jpg").convert()
    tux = pygame.image.load("tux.png").convert_alpha()

    # Indicamos la posicion de las "Surface" sobre la ventana
    screen.blit(fondo, (0, 0))
    screen.blit(tux, (550, 200))
    # se muestran lo cambios en pantalla
    pygame.display.flip()
```

Si se fijan al cargar el fondo se uso fondo = pygame.image.load("fondo.jpg").convert() mientras que para la imagen de tux se uso tux = pygame.image.load("tux.png").convert_alpha(). Esto se debe a que el fondo no necesita tener un color trasparente (un canal alpha) ya que es la imagen que esta bajo todas las demás, en cambio el sprite de nuestro pingüino si necesita tener un color trasparente (que convenientemente ya esta definido en el png ;), lo hice a propósito cuando lo cree) o de lo contrario al cargar la imagen del pingüino se vería un feo rectángulo a su alrededor.

Luego se hizo screen.blit(fondo, (0, 0)) y screen.blit(tux, (550, 200)), con esto se le indica que cargue la imagen del fondo justo en la coordenada (0,0) para que cubra toda la pantalla (cualquier otro valor hubiera dejado el fondo desplazado) y en el caso de tux se le indica que lo cargue a la derecha.

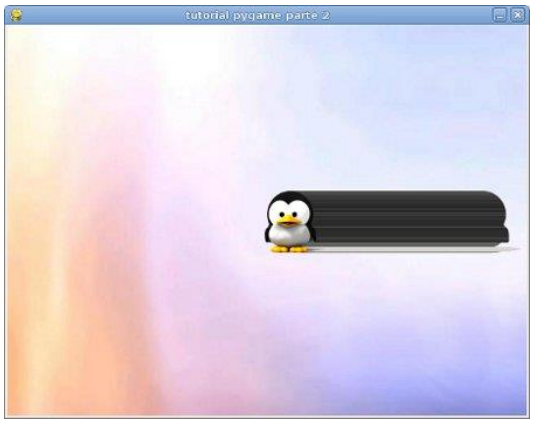
Paso 4: Moviendo la imagen

Ahora vamos a hacer que tux se mueva a la izquierda, para ello, simplemente a su posición en el eje x le vamos a restar -1 en cada ciclo del bucle principal. Para ello modificamos el código para que quede de la siguiente manera:

```
    # Indicamos la posicion de las "Surface" sobre la ventana
    screen.blit(tux, (tux_pos_x, tux_pos_y))
    screen.blit(fondo, (0, 0))
    # se muestran lo cambios en pantalla
    pygame.display.flip()

    # el bucle principal del juego
    while True:
        # le restamos 1 a la coordenada x de tux
        # asi se mueve un poquito a la izquierda
        tux_pos_x = tux_pos_x - 1
        screen.blit(tux, (tux_pos_x, tux_pos_y))
        # se muestran lo cambios en pantalla
        pygame.display.flip()

        # Posibles entradas del teclado y mouse
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                sys.exit()
```



¿Y qué paso?

Explicación: lo que paso es que se nos olvidó borrar al pingüino de la pantalla antes de moverlo a su nueva posición, por lo que veremos en realidad un "rastro" del movimiento mientras dibujamos continuamente al pingüino en nuevas posiciones.

Además, nunca le pusimos limite al movimiento del pingüino, por lo que una vez que alcanzo el borde de la pantalla, siguió moviéndose hacia la izquierda.

Paso 5 (final): Refrescando/borrando la pantalla

La solución más fácil para borrar el "rastro" que se deja al moverse, es simplemente redibujar toda la pantalla (fondo incluido) en cada ciclo del juego, por lo tanto dentro del while, después de actualizar la posición del pingüino, se redibuja tota la pantalla usando blit() y pygame.display.flip().

```
# el bucle principal del juego
    while True:
        # le restamos 1 a la coordenada x de tux y comprobamos
        # que no alcance el borde de la pantalla
        tux_pos_x = tux_pos_x - 1
        if tux_pos_x < 1:
            tux_pos_x = 550

        # Redibujamos todos los elementos de la pantalla
        screen.blit(fondo, (0, 0))
        screen.blit(tux, (tux_pos_x, tux_pos_y))
        # se muestran lo cambios en pantalla
        pygame.display.flip()
```

Además, después de actualizar la tux_pos_x se colocó un if, de tal manera de que si esa posición es 0 o negativa (o sea esta fuera de la pantalla), la tux_pos_x tome un valor de 550 (550 + 90px que mide de ancho la imagen de tux = 640 px, o sea que la imagen se muestre al borde derecho de la pantalla).

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
```

```
# Escrito por Daniel Fuentes B.
# Licencia: X11/MIT license http://www.opensource.org/licenses/mit-license.php
# http://pythonmania.wordpress.com/2010/03/25/tutorial-pygame-2-ventana-e-imagenes/

# ---------------------------
# Importacion de los módulos
# ---------------------------

import pygame
from pygame.locals import *
import sys

# -----------
# Constantes
# -----------

SCREEN_WIDTH = 640
SCREEN_HEIGHT = 480


def main():
    pygame.init()
    # creamos la ventana y le indicamos un título:
    screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
    pygame.display.set_caption("MOVIENDO A TUTO")

    # cargamos el fondo y una imagen (se crea objetos "Surface")
    fondo = pygame.image.load("fondo.jpg").convert()
    tux = pygame.image.load("tux.png").convert_alpha()

    tux_pos_x = 550
    tux_pos_y = 200

    # Indicamos la posicion de las "Surface" sobre la ventana
    screen.blit(tux, (tux_pos_x, tux_pos_y))
    screen.blit(fondo, (0, 0))
    # se muestran lo cambios en pantalla
    pygame.display.flip()

    # el bucle principal del juego
    while True:
        # le restamos 1 a la coordenada x de tux y comprobamos
        # que no alcance el borde de la pantalla
        tux_pos_x = tux_pos_x - 1
        if tux_pos_x < 1:
            tux_pos_x = 550

        # Redibujamos todos los elementos de la pantalla
        screen.blit(fondo, (0, 0))
        screen.blit(tux, (tux_pos_x, tux_pos_y))
        # se muestran lo cambios en pantalla
        pygame.display.flip()

        # Posibles entradas del teclado y mouse
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                sys.exit()


if __name__ == "__main__":
    main()
```

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

# Autor:
# Licencia: X11/MIT license http://www.opensource.org/licenses/mit-license.php
# Fecha:

# ---------------------------
# Importación de los módulos
# ---------------------------
import pygame
from pygame.locals import *
import sys
import math

# -----------
# Constantes
# -----------
WIDTH = 640
HEIGHT = 480

# ------------------------------
# Clases y Funciones utilizadas
# ------------------------------
class Proyectil(pygame.sprite.Sprite):
    "Clase que representa el proyectil lanzado"

    def __init__(self, x, y):
        self.angulo = 45
        self.veloc = 50
        self.tiempo = 0
        self.x = x
        self.y = y
        self.disparar = False
        self.xreal = x
        self.yreal = HEIGHT - self.y

    def update(self):
        "actualizar la posición del proyectil"
        self.velocx = self.veloc * math.cos(math.radians(self.angulo))
        self.velocy = self.veloc * math.sin(math.radians(self.angulo))

        if self.disparar == True:
            # esta en movimiento, hay que actualizar la posición
            self.xreal = (0 + self.velocx * self.tiempo)
            self.yreal = (0 + self.velocy * self.tiempo +
                (-9.8 * (self.tiempo ** 2)) / 2)
            # Corregir la posición en el eje vertical
            self.x = self.xreal
            self.y = HEIGHT - self.yreal
        else:
            # se mantiene sin disparar, por lo cual no se hace nada
            pass

        # si sale de la pantalla reiniciar la posición (a inferior izq.)
        if (self.y > HEIGHT) or (self.x > WIDTH):
            self.x = 0
            self.y = HEIGHT
            self.tiempo = 0
            self.disparar = False

# ------------------------------
# Función principal del juego
# ------------------------------

def main():
    pygame.init()
    # creamos la ventana y le indicamos un titulo:
```

```
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("tutorial pygame parte 4")

    # se define la letra por defecto
    fuente = pygame.font.Font(None, 20)

    # se crea un proyectil a lanzar
    bala = Proyectil(0, HEIGHT)

    pygame.key.set_repeat(1, 80)  # Activa repetición de teclas
    clock = pygame.time.Clock()

    # el bucle principal del juego
    while True:
        # registramos cuanto ha pasado desde el ultimo ciclo
        tick = clock.tick(60)
        # Posibles entradas del teclado y mouse
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                sys.exit()
            elif event.type == pygame.KEYDOWN:
                if event.key == K_UP:
                    if bala.angulo < 90 and bala.disparar == False:
                        bala.angulo = bala.angulo + 1
                elif event.key == K_DOWN:
                    if bala.angulo > 0 and bala.disparar == False:
                        bala.angulo = bala.angulo - 1
                elif event.key == K_RIGHT:
                    if bala.veloc < 100 and bala.disparar == False:
                        bala.veloc = bala.veloc + 1
                elif event.key == K_LEFT:
                    if bala.veloc > 10 and bala.disparar == False:
                        bala.veloc = bala.veloc - 1
                elif event.key == K_SPACE:
                    bala.disparar = True
                elif event.key == K_ESCAPE:
                    sys.exit()

        if bala.disparar == True:
            # al tiempo anterior le sumamos lo transcurrido
            bala.tiempo = bala.tiempo + (tick / 1000.0)

        # Actualizar la posición e información
        bala.update()
        text = "Velocidad: %3d (m/s)   Angulo: %d   x=%d m   y=%d m" % (
            bala.veloc, bala.angulo, bala.xreal, bala.yreal)
        mensaje = fuente.render(text, 1, (255, 255, 255))

        # Re dibujar los elementos en pantalla
        screen.fill((30, 145, 255))
        screen.blit(mensaje, (15, 5))
        pygame.draw.line(screen, (255, 255, 255), (0, 25), (640, 25), 2)
        pygame.draw.circle(screen, (0, 0, 0), (int(bala.x), int(bala.y)), 10)
        # actualizamos la pantalla
        pygame.display.flip()


if __name__ == "__main__":
    main()
```